

[75] Vector-db-benchmark: Open-source Benchmark Suite for Vector Database Performance

요약

myscale(2024)의 오픈소스 벤치마크 스위트로, 벡터 데이터베이스 성능을 체계적으로 측정하고 비교할 수 있는 도구를 제공한다. 벡터 검색 시장의 급속한 성장으로 수많은 벡터 DB가 등장했으나, 성능 비교를 위한 표준화된 평가 방식이 부재했다.

본 벤치마크는 다양한 벡터 DB (Milvus, Weaviate, Pinecone, Vespa 등)와 데이터셋을 지원하여 공정한 비교를 가능하게 한다. 핵심 메트릭은:

- **throughput (QPS):** 초당 처리 쿼리 수
- **latency:** 개별 쿼리 응답 시간
- **recall:** 검색 정확도 (top-k 일치도)
- **memory:** 메모리 사용량
- **indexing_time:** 인덱스 구축 시간

벤치마크 설계의 특징은 실제 사용 시나리오를 반영하는 쿼리 믹스, 다양한 데이터 특성(차원, 크기, 분포), 그리고 필터링과 같은 추가 조건을 지원한다는 점이다.

본 논문과의 관계: Exqutor의 성능 평가에 필수적인 벤치마크 프레임워크를 제공한다. 범위 필터와 벡터 검색의 결합 조건에서의 성능을 측정할 수 있는 기반을 제공하며, 선택도 추정의 정확도가 최종 쿼리 성능에 미치는 영향을 정량화할 수 있다.

상세분석

75.1 벤치마크 설계 철학

공정성(Fairness) 원칙:

모든 시스템이 동등한 조건에서 테스트:

- 데이터 동일
- 쿼리 동일
- 메모리 제약 동일
- 하드웨어 동일
- 충분한 워밍업(warm-up) 시간 제공

실제성(Realism) 추구:

- 합성 데이터가 아닌 실제 임베딩 데이터 사용
- 실제 검색 쿼리 패턴 모델링
- 동시 사용자 부하 시뮬레이션

확장성(Scalability) 검증:

- 데이터 크기별 성능 측정 (1M ~ 10B 벡터)
- 차원별 성능 변화 관찰
- 메모리 제약 하에서의 성능 곡선

75.2 지원 데이터셋

공개 데이터셋:

데이터셋	벡터 수	차원	특징
SIFT-1M	1M	128	자연 영상 특징
GIST-1M	1M	960	넓은 영상 특징
Deep1B	1B	96	심층 학습 기반
OpenAI embeddings	1M	1536	텍스트 임베딩
Cohere embeddings	1M	4096	고차원 텍스트

데이터셋 특성:

- **자연 데이터:** 실제 이미지/텍스트 임베딩
- **합성 데이터:** 다양한 분포(uniform, gaussian, clustered) 제어 가능
- **스케일:** 소형(1M)부터 대형(10B)까지

75.3 성능 메트릭 정의

3.1 Recall (정확도)

$\text{Recall}@k = |\text{실제 top-}k \cap \text{반환된 top-}k| / k$

예: 실제 top-10 은 [1,2,3,4,5,6,7,8,9,10]

반환된 top-10 은 [1,2,3,4,5,11,12,13,14,15]

$\text{Recall}@10 = 5/10 = 50\%$

3.2 QPS (Query Per Second)

$\text{QPS} = \text{총 쿼리 수} / \text{측정 시간(초)}$

측정 조건:

- 단일 쓰레드 QPS (기준선)
- 멀티 쓰레드 QPS (병렬성 측정)
- 부하 곡선 (동시 클라이언트 증가에 따른 QPS 변화)

3.3 Latency (응답 시간)

p50_latency: 중앙값 응답 시간

p95_latency: 상위 5% 응답 시간 (꼬리 지연)

p99_latency: 상위 1% 응답 시간

시스템 품질 판단:

- p50: 일반적 성능
- p95/p99: 안정성 (SLA 확보)

3.4 Throughput-Latency 트레이드오프

높은 QPS = 낮은 latency? (반드시 아님)

└─ 배치 처리로 높은 처리량 얻을 수 있으나 개별 지연 증가

3.5 메모리 사용량

인덱싱 메모리: 구축 중 최대 메모리

서빙 메모리: 쿼리 처리 중 메모리 (인덱스 + 버퍼)

메모리 효율: QPS / GB (단위 메모리당 처리량)

75.4 쿼리 유형 및 시나리오

기본 쿼리 유형:

4.1 벡터 유사성 검색

```
query = {
  "vector": [0.1, 0.2, ..., 0.5], # d-차원 벡터
  "top_k": 10,
  "metric": "cosine" # or "l2", "inner_product"
}
```

4.2 범위 필터 + 벡터 검색

```
query = {
  "vector": [0.1, 0.2, ..., 0.5],
  "top_k": 10,
  "filter": {
    "price": {"$lt": 100},
    "category": "electronics"
  }
}
```

4.3 메타데이터 필터만

```
query = {
  "filter": {
    "date": {"$gte": "2024-01-01"},
    "status": "active"
  }
}
# 벡터 검색 없이 필터링만
```

4.4 하이브리드 검색

```
query = {
  "vector_search": {...},
  "keyword_search": {...},
  "combine": "linear_combination" # 가중 조합
}
```

75.5 벤치마크 실행 워크플로우

단계 1: 초기화

데이터셋 다운로드 / 생성
벡터 DB 인스턴스 시작 (컨테이너 기반)
메모리, CPU 할당 확인

단계 2: 인덱싱

벡터들을 DB 에 로드
다양한 인덱싱 파라미터로 여러 설정 시도
|— HNSW M=16, ef=200
|— HNSW M=8, ef=100
|— IVF nlist=1000, nprobe=10
|— ...

단계 3: 워밍업

100 회 쿼리 실행 (캐시 워밍)
메모리 할당 안정화
스케줄러 스트레스 해제

단계 4: 벤치마크 실행

10,000~100,000 개 쿼리 실행
각 쿼리의 latency, recall 기록

단계 5: 결과 분석

QPS, p50/p95/p99 latency 계산
Recall vs Latency 곡선 그리기
메모리 효율 계산

75.6 결과 예시 및 해석

출력 형식:

```
System: Milvus 2.3
Dataset: SIFT-1M
Metric: L2

Configuration: index_type=HNSW, M=16, ef_construction=200, ef_search=10

Result:
QPS: 1,200 req/s
Latency (p50): 0.8 ms
Latency (p95): 2.5 ms
Latency (p99): 5.0 ms
Recall@10: 98.5%
Memory: 850 MB
Memory efficiency: 1.41 QPS/MB
```

결과 해석:

- QPS 높음(1200): 처리 능력 우수
- p99 latency 낮음(5ms): SLA 만족 가능
- recall 높음(98.5%): 정확도 우수
- 메모리 효율 좋음(1.41): 메모리당 처리량 많음

75.7 필터링 시나리오 벤치마크

필터 선택도의 영향:

필터 선택도 10% (10,000 개 벡터만 매칭):
QPS: 1,800 req/s (필터만으로 90% 제거)
Recall@10: 95%

필터 선택도 50% (500,000 개 벡터 매칭):
QPS: 950 req/s
Recall@10: 95%

필터 선택도 100% (모든 벡터 매칭):
QPS: 800 req/s
Recall@10: 95%

발견:

- 선택도가 낮을수록 QPS 향상 (필터로 대부분 제거)
- 선택도 추정의 정확성이 중요 (과소/과다 추정 시 성능 저하)

75.8 본 논문과의 관계

Exqutor 성능 평가:

Vector-db-benchmark 는 다음과 같은 평가를 가능하게 한다:

1. 기본 성능 측정:

- 범위 필터 없음: 기준선 QPS
- 범위 필터 있음: 필터 적용 시 QPS
- 성능 저하율: (기준선 QPS - 필터 있음 QPS) / 기준선 QPS

1. 선택도 추정의 영향:

정확한 선택도 추정 → 최적 실행 계획 → 높은 성능
부정확한 선택도 추정 → 나쁜 실행 계획 → 낮은 성능

1. 메모리 제약 시 성능:

Exqutor 는 제한된 메모리 내에서:
- 선택도 추정 모델
- 벡터 인덱스
- 메타데이터 캐시
를 관리해야 함

추가 제기 문제

1. **우편향 평가:** 특정 DB 에 유리한 데이터셋이나 쿼리 패턴은 없는가?
2. **실시간 업데이트 벤치마크:** 인덱싱 중 동시 쿼리 처리 성능은?
3. **멀티테넌시 시나리오:** 여러 사용자의 동시 쿼리로 인한 성능 간섭은?
4. **캐시 효과 통제:** 운영 체제 캐시로 인한 성능 향상을 어떻게 통제할 것인가?
5. **선택도 추정 정확도의 영향:** 선택도 추정 오류가 최종 성능에 얼마나 영향을 미치는가?
6. **네트워크 레이턴시:** 원격 벡터 DB 접근 시 네트워크 지연의 영향은?